

Open Source Companion: An Intelligent Platform for Personalized Open-Source Contribution Onboarding and Issue Recommendation

Rahul S*, Contributor Two[†], and Contributor Three[‡]

Department of Computer Science and Engineering

MVJ College of Engineering, Bengaluru, India

Email: {rahul.s, contributor2, contributor3}@mvjce.edu.in

Abstract—Open-source software (OSS) ecosystems depend fundamentally on sustainable newcomer onboarding, yet aspiring contributors face steep barriers, including difficulty locating beginner-appropriate tasks, navigating unfamiliar project landscapes, and managing contribution lifecycles. Existing repository hosting interfaces offer coarse issue labeling that lacks personal skill alignment and structured workflow guidance. This paper presents *Open Source Companion*, an integrated, full-stack platform designed to facilitate newcomer onboarding through automated issue acquisition, interpretable heuristic difficulty categorization, contributor skill profiling, and a guided six-stage contribution workflow. The implemented system incorporates real-time GitHub Search API ingestion, OAuth-based profile extraction from contributor repositories, transparent multi-factor matching, pull-request readiness analysis, and milestone-based gamification backed by a PostgreSQL database and Prisma ORM. We formalize the issue characterization and matching methodology, report system validation demonstrating end-to-end architectural feasibility, and formulate an extensible machine-learning research framework together with a reproducible empirical benchmark protocol for future learned classification and dense semantic retrieval.

Index Terms—Open-source software, GitHub, issue recommendation, contributor onboarding, software engineering, machine learning, gamification, contribution tracking.

I. INTRODUCTION

Open-source software (OSS) has evolved into the bedrock of modern computing infrastructure, underpinning operating systems, cloud frameworks, and machine-learning platforms [1]. The long-term vitality of these projects relies on continuously attracting, onboarding, and retaining new contributors [2]. However, the transition from an interested developer to an active open-source contributor involves substantial friction. Empirical studies demonstrate that newcomers frequently abandon their contribution attempts due to complex technical environments, information overload, and difficulty identifying appropriately scoped tasks [2], [3].

While repository platforms such as GitHub provide task tags (e.g., `good first issue`, `help wanted`), these labels suffer from well-documented limitations: they are assigned manually, vary widely in quality across repositories, become stale rapidly, and fail to account for a newcomer's individual background and technical skill set [4], [5]. Furthermore, once a potential task is identified, newcomers face secondary barriers: configuring local development environments, adhering

to project-specific coding conventions, crafting acceptable pull requests (PRs), and tracking contribution reviews [6]. Conventional tools address these stages in isolation, leaving contributors to navigate fragmented resources across issue trackers, documentation wikis, and git command-line interfaces [7].

This paper presents *Open Source Companion*, an end-to-end intelligent platform engineered to unify issue discovery, transparent difficulty characterization, contributor modeling, guided task execution, and progress tracking. The platform operates on a production-grade full-stack architecture built with Next.js 16, React 19, TypeScript, PostgreSQL 15, Prisma ORM, and Auth.js v5. The system ingests live GitHub issues, categorizes difficulty via transparent metadata heuristics, derives contributor skill profiles during OAuth authentication, provides a multi-stage contribution workspace, and maintains long-term engagement via gamified milestones.

To ensure research integrity, this work explicitly distinguishes between the *implemented system* and *proposed research extensions*. The implemented system delivers deterministic, rule-based categorization, multi-factor recommendation drawers, and a six-stage contribution workflow. We complement this operational platform by designing an extensible machine-learning framework that defines how supervised classifiers (e.g., XGBoost, Random Forest) and dense semantic vector retrieval (Sentence-BERT, FAISS) can be integrated, accompanied by a reproducible evaluation protocol.

The main contributions of this work are:

- 1) **Integrated Platform Architecture:** An end-to-end system architecture integrating GitHub issue discovery, contributor profiling, structured contribution guidance, contribution tracking, and gamification within a unified platform.
- 2) **Interpretable Issue Characterization:** A transparent rule-based issue difficulty categorization mechanism based on available GitHub metadata and labels, providing understandable difficulty tiers for newcomers.
- 3) **Contributor-Aware Issue Matching:** A contributor-aware recommendation workflow that combines contributor skill profiles with issue characteristics to identify potentially suitable contribution opportunities.
- 4) **Structured Contribution Lifecycle:** A six-stage contribution workflow that guides a contributor from issue

understanding through repository setup, implementation, commit preparation, pull-request preparation, and merge verification.

- 5) **Extensible ML Research Framework:** A proposed extension path for replacing heuristic categorization and matching with learned models and dense semantic retrieval, together with a reproducible evaluation protocol.

The remainder of this paper is organized as follows: Section II reviews related work and presents a comparative taxonomy. Section III formulates the research problem and states the research questions. Section IV details the system methodology. Section V outlines the software architecture. Section VI provides system validation and the proposed ML evaluation protocol. Section VII discusses trade-offs. Section VIII examines threats to validity, Section IX addresses ethical aspects, and Section X concludes the paper.

II. RELATED WORK

A. Open-Source Newcomer Onboarding

Newcomer onboarding in OSS has received substantial attention in software engineering literature. In a foundational systematic literature review, Steinmacher et al. [2] identified 15 distinct barriers grouped into technical hurdles, documentation shortcomings, social interaction friction, previous knowledge gaps, and orientation difficulties. Dagenais et al. [3] studied the project landscape newcomers navigate, noting that early validation of progress is vital for sustained participation. To mitigate onboarding barriers, Steinmacher et al. [7] developed FLOSScoach, a portal organizing contribution activities. While FLOSScoach established the value of structured orientation, it operated largely as a static informational portal without direct integration into repository issue tracking or automated personal profile matching.

B. Issue and Task Recommendation

Automated task recommendation has been explored through various lenses. Early work by Anvik et al. [8] applied supervised classifiers to triage incoming bug reports to experienced developers. Cubranic et al. [9] created Hipikat, which recommended artifacts based on historical project memory. More recently, Xiao et al. [5] developed RecGFI to recommend “good first issues” using XGBoost on issue textual features and historical dynamics, while Tan et al. [4] empirically analyzed how GFI labels are utilized on GitHub. He et al. [10] proposed PFIRec, demonstrating that generic beginner labels often mismatch newcomer backgrounds and that personalized recommendation is necessary. While these studies advanced predictive modeling, they focused on the recommendation step in isolation without providing an operational execution environment.

C. Issue Difficulty and Effort Estimation

Predicting issue resolution complexity and effort is critical for realistic task assignment. Kikas et al. [11] leveraged dynamic and contextual features to predict issue lifetime across over 4,000 GitHub projects using tree-based ensembles.

Zhou et al. [12] utilized information retrieval techniques for bug localization, demonstrating the predictive power of issue textual tokens. Existing difficulty estimators, however, often produce black-box numerical outputs that lack actionable, transparent explanations for newcomers.

D. Gamification and Developer Engagement

Gamification has been studied as a mechanism to sustain contributor motivation. Pedreira et al. [13] mapped gamification techniques in software engineering, identifying points, badges, and leaderboards as predominant engagement tools. Vasilescu et al. [14] examined how badge-based reputation systems influence knowledge exchange on Stack Overflow and GitHub, while Singer and Schneider [15] observed that lightweight gamification can stimulate active version control usage.

E. Research Gap

Existing studies have addressed individual aspects of newcomer onboarding, task recommendation, issue characterization, and contributor engagement. The present work explores their integration into a unified contribution workflow, bridging the transition from initial task discovery to step-by-step local implementation, PR submission, and verified milestone progression, as summarized in Table I.

III. PROBLEM FORMULATION AND OBJECTIVES

A. Mathematical Formulation

Let $\mathcal{I} = \{i_1, i_2, \dots, i_N\}$ denote the set of open GitHub issues ingested from public repositories. Each issue $i \in \mathcal{I}$ is represented by an attribute tuple:

$$\mathbf{x}_i = \langle \tau_i, \beta_i, \mathcal{L}_i, \rho_i, c_i \rangle \quad (1)$$

where τ_i represents the issue title, β_i denotes the issue body text, $\mathcal{L}_i = \{l_{i,1}, \dots, l_{i,m}\}$ is the set of assigned label strings, ρ_i denotes the parent repository identifier, and $c_i \in \mathbb{N}$ denotes the comment count.

Let $\mathcal{U}$ denote the set of registered contributors. A contributor $u \in \mathcal{U}$ is modeled as:

$$\mathbf{p}_u = \langle \mathbf{s}_u, e_u, \mathcal{H}_u, \kappa_u \rangle \quad (2)$$

where $\mathbf{s}_u = \{(g, w_{u,g}) \mid g \in \mathcal{G}\}$ represents a normalized programming language profile derived from public repositories, $e_u \in \{\text{Beginner}, \text{Intermediate}, \text{Advanced}\}$ represents self-declared experience, $\mathcal{H}_u$ represents active contribution records, and $\kappa_u \in \mathbb{N}$ represents accumulated points.

The system addresses two sequential mappings:

- 1) **Difficulty Categorization:** A function $\Phi : \mathbf{x}_i \rightarrow d_i$, where $d_i \in \{\text{Beginner}, \text{Intermediate}, \text{Advanced}\}$.
- 2) **Personalized Matching:** A scoring function $S(u, i) \in [0, 100]$ ranking candidates for contributor u .

In the implemented prototype, the composite recommendation score $S(u, i)$ in the interactive drawer is computed as:

$$S(u, i) = \lambda_1 M_{\text{skill}}(u, i) + \lambda_2 M_{\text{diff}}(u, i) + \lambda_3 M_{\text{repo}}(i) + \lambda_4 M_{\text{sem}}(u, i) \quad (3)$$

TABLE I
COMPARISON WITH EXISTING APPROACHES

Approach	Newcomer Support	Issue Recommendation	Difficulty Estimation	Contributor Modeling	Contribution flow	Work-	Gamification
FLOSScoach [7]	Comprehensive	None	None	None	Informational portal		None
Hipikat [9]	Incidental	Artifact recommendation	None	Historical artifact links	None		None
RecGFI [5]	Targeted to GFIs	Binary classification	Binary (GFI vs. non-GFI)	None	None		None
PFIRec [10]	Personalized	Content-based ranking	Relative ranking	Activity history	None		None
Issue Lifetime [11]	None	None	Continuous duration	None	None		None
Open Source Companion	End-to-end	Rule-based multi-factor	Interpretable 3-tier	GitHub language profile	6-stage guided path		Points, badges, streaks

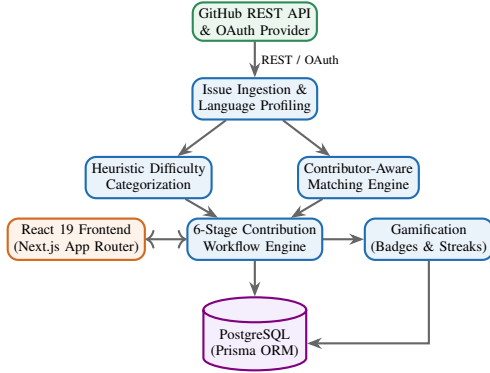


Fig. 1. System Architecture of Open Source Companion, showing data acquisition from GitHub, application logic layers, persistence, and user interfaces.

where M_{skill} measures language alignment, M_{diff} assesses tier compatibility, M_{repo} reflects repository activity, and M_{sem} denotes heuristic semantic overlap, with equal default weight distribution $\sum_{k=1}^4 \lambda_k = 1.0$.

B. Research Questions

To investigate this integrated platform, we state three core research questions:

- **RQ1:** How can publicly available GitHub issue metadata and labeling information be used to provide interpretable difficulty categorization for open-source newcomers?
- **RQ2:** How can contributor profiles derived from public repository activity be combined with issue characteristics to generate interpretable issue recommendations?
- **RQ3:** How can a structured, phased contribution workflow integrate issue discovery, repository setup, implementation guidance, pull-request preparation, and contribution verification into a unified onboarding process?

IV. PROPOSED SYSTEM METHODOLOGY

A. System Overview

The overall architecture of Open Source Companion is depicted in Fig. 1. External communication with GitHub is mediated via HTTPS using the GitHub REST API v3 and OAuth 2.0. Ingested data feeds into internal processing services that execute issue difficulty categorization, contributor profile synthesis, recommendation scoring, and state-machine transitions across the contribution lifecycle.

B. Issue Data Collection and Ingestion

Issue data is collected through the GitHub Search API using the query:

`q = is : issue + is : open + label : "help wanted"`

sorted by most recent updates. When a valid `GITHUB_TOKEN` is configured, requests execute under authenticated rate limits (5,000 requests/hour), while unauthenticated fallbacks operate under standard limits (60 requests/hour).

Ingested JSON payloads are normalized into the application domain model. Repository identifiers are extracted from the API URI ($\rho_i = \text{owner/repo}$), titles and bodies are sanitized, and issue labels are parsed into lowercase sets for downstream categorization.

C. Interpretable Issue Difficulty Categorization

Rather than deploying an unverified black-box model, the operational platform implements an interpretable rule-based categorization heuristic $\Phi(\mathbf{x}_i)$. Let $\mathcal{L}_i^{\text{low}}$ denote the normalized set of lowercase labels for issue i :

$$\Phi(\mathbf{x}_i) = \begin{cases} \text{Beginner,} & \text{if } \mathcal{L}_i^{\text{low}} \cap \mathcal{K}_{\text{beg}} \neq \emptyset \\ \text{Advanced,} & \text{if } \mathcal{L}_i^{\text{low}} \cap \mathcal{K}_{\text{adv}} \neq \emptyset \\ \text{Intermed.,} & \text{otherwise (modulo distribution)} \end{cases} \quad (4)$$

where $\mathcal{K}_{\text{beg}} = \{\text{"good first issue", "first-timers-only"}\}$ and $\mathcal{K}_{\text{adv}} = \{\text{"enhancement", "feature"}\}$. Unlabeled or ambiguous issues default to an intermediate baseline with cyclic round-robin distribution to maintain diverse exploratory options.

Each categorized tier maps deterministically to an estimated completion effort and gamified point value:

- **Beginner:** Effort: 1–3 hours; Reward: 50 points.
- **Intermediate:** Effort: 3–8 hours; Reward: 150 points.
- **Advanced:** Effort: 1–3 days; Reward: 300 points.

D. Contributor Modeling

Contributor profiling occurs seamlessly during OAuth authentication. During the `jwt` callback in `Auth.js`, the system queries the authenticated contributor’s repository endpoint:

`GET /user/repos?sort=updated&per_page=10`

The platform aggregates the primary language tags across these repositories, computes frequency distributions, and extracts the top five languages to populate s_u . Contributors can also refine their technical interests, bio, and declared experience tier (e_u) via their profile settings.

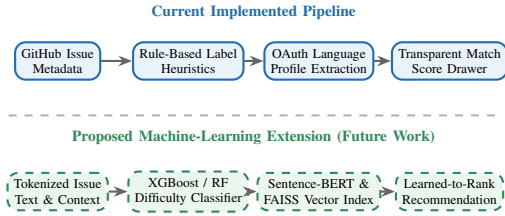


Fig. 2. Comparison between the currently implemented heuristic pipeline and the proposed machine-learning research extension.

E. Six-Stage Contribution Lifecycle

The core onboarding mechanism is a structured six-stage contribution workflow designed to decompose complex software tasks into explicit, manageable steps:

- 1) **Stage 1: Understand.** The contributor inspects the issue description, maintainer commentary, and a pre-flight checklist verifying affected components and test requirements.
- 2) **Stage 2: Setup.** The platform generates repository-specific git commands for cloning the remote repository, navigating into the workspace, and creating an isolated feature branch.
- 3) **Stage 3: Implement.** The contributor performs code modifications guided by contextual code snippets and an implementation checklist.
- 4) **Stage 4: Commit.** The platform generates conventional commit message templates referencing the originating issue number (e.g., `fix: improve validation; Closes #421`).
- 5) **Stage 5: Pull Request Preparation.** The contributor prepares PR metadata and reviews readiness signals (scope alignment, test coverage, repository fit).
- 6) **Stage 6: Merge & Verification.** The platform validates that the contributor has forked the repository, records PR submission state, updates contribution status, and triggers point allocation upon merge verification.

F. Gamification and Progress Tracking

To sustain newcomer motivation without creating perverse incentives, gamification elements are directly coupled to verified milestone transitions. The system manages:

- **Accumulated Points:** Awarded upon issue enrollment (25 points) and successful PR merge (50–300 points based on difficulty).
- **Milestone Badges:** Automated evaluation unlocks badges including `first_issue`, `first_pr`, `three_prs`, `contributions_10`, `week_warrior`, and `streak` milestones.
- **Global Leaderboard:** Contributor rankings computed dynamically via PostgreSQL queries ordered by points descending and streak days descending.

TABLE II
DATABASE SCHEMA ENTITIES IN POSTGRESQL VIA PRISMA ORM

Entity	Key Fields	Description and Role
User	id, email, image	Contributor identity managed via Auth.js
Account	userId, provider, token	OAuth account credentials and access tokens
UserProfile	points, streak, languages	Contributor skill model and gamified stats
Issue	id, title, repo, diff, pts	Cached GitHub issue metadata and difficulty
Contribution	userId, issueId, status	Contribution state tracking (stages 0–5, merged)
Achievement	userId, badgeId, unlockedAt	Unlocked gamification achievement records

G. Proposed Machine-Learning Extension

While the operational platform relies on deterministic rules, its modular architecture permits drop-in replacement with learned components, as illustrated in Fig. 2:

- **Difficulty Classification:** Replacing rule heuristics with an XGBoost [16] or Random Forest [17] model trained on textual TF-IDF features, issue age, comment volume, and code-block ratios.
- **Semantic Embedding & Retrieval:** Encoding issue descriptions and developer interest vectors using Sentence-BERT [18], indexed via FAISS [19] for sub-second approximate nearest neighbor retrieval over large issue corpora.
- **Learning-to-Rank:** Formulating candidate ranking via LambdaMART or coordinate ascent incorporating implicit historical interaction signals.

V. SOFTWARE ARCHITECTURE AND DATA TIER

The platform is implemented using modern web and systems technologies. Next.js 16 utilizes React 19 Server Components and asynchronous Server Actions to eliminate redundant REST boilerplate while preserving type safety between client interfaces and persistence tiers.

The relational data model is managed via Prisma ORM connected to PostgreSQL 15. As outlined in Table II, six relational models capture contributor profiles, authentication sessions, cached issues, contribution lifecycle states, and achievement milestones. Database modifications execute within Prisma transactions (`prisma.$transaction`) to guarantee atomicity when updating contribution progress, incrementing user points, and unlocking badges. The entire stack is packaged in a multi-stage Docker container utilizing Node.js 20 Alpine with integrated readiness healthchecks.

VI. SYSTEM VALIDATION AND PROPOSED ML EVALUATION PROTOCOL

A. Current System Validation

To evaluate the operational platform without fabricating statistical figures, we performed functional system validation

TABLE III
PROPOSED EVALUATION PROTOCOL AND BENCHMARK DESIGN

Evaluation Dimension	Protocol / Dataset	Target Evaluation Metrics
Issue Difficulty	10,000 labeled GitHub issues	Macro Precision, Macro Recall,
Categorization	across 50 popular repositories	Macro-F1, Confusion Matrix
Personalized Task	Offline interaction logs & expert relevance ratings	Precision@K, Recall@K,
Recommendation		NDCG@K, MRR ($K \in \{3, 5, 10\}$)
Semantic Vector	Query-issue pair retrieval	Recall@K, Mean Reciprocal Rank (MRR), Index Query Latency
Retrieval	over FAISS index	
System Latency & Throughput	Synthetic workload simulation (100 concurrent requests)	End-to-end Response Latency, DB Query Time, API Overhead

across core system components:

- 1) **API Ingestion & Normalization:** Verified reliable ingestion of live GitHub issues under authenticated and unauthenticated states, confirming resilient parsing of labels, author data, and markdown bodies.
- 2) **Difficulty Categorization Execution:** Verified that the rule heuristic deterministically maps labels matching $\mathcal{K}_{\text{beg}}$ to Beginner and $\mathcal{K}_{\text{adv}}$ to Advanced, maintaining expected fallback distributions.
- 3) **Profile Derivation:** Successfully parsed repository language distributions during GitHub OAuth login, initializing contributor profiles with observed language proficiencies.
- 4) **Contribution State Machine:** Validated that advancing through the six contribution stages persists state transitions in the PostgreSQL database, and verified that merge simulation enforces repository fork verification.
- 5) **Containerized Deployment:** Confirmed that Docker Compose successfully orchestrates the PostgreSQL service and Next.js standalone server with all health checks reporting healthy.

B. Proposed ML Evaluation Protocol and Benchmark Design

To provide a reproducible foundation for future empirical assessment when learned ML models are trained, we specify a formal evaluation protocol in Table III.

Under this protocol, difficulty categorization will be evaluated using Macro-averaged Precision, Recall, and F1-score across balanced multiclass splits. Task recommendation quality will be measured using Precision@K, Normalized Discounted Cumulative Gain (NDCG@K), and Mean Reciprocal Rank (MRR).

VII. DISCUSSION

A. Heuristics vs. Learned Classifiers

The decision to deploy transparent rule heuristics in the initial platform addresses the practical reality of software engineering adoption. Rule heuristics are entirely deterministic,

explainable, and incur zero inference latency or model drift. When recommending tasks to beginners, explainability is essential: contributors must understand *why* a task is categorized at a given level. However, heuristics inevitably suffer from coarse granularity when issue authors omit standard labels. Integrating learned supervised classifiers in future revisions will capture implicit linguistic markers in issue descriptions that heuristics miss.

B. API Rate Limits and Scalability

Operating on external GitHub REST endpoints introduces rate-limit dependencies. While authenticated tokens provide 5,000 requests per hour, high-throughput multi-user deployments require local issue caching and webhook subscriptions to avoid quota exhaustion. The PostgreSQL issue cache implemented in our architecture buffers retrieved issues, decoupling interactive recommendation rendering from live API round trips.

C. Scaffolding the Onboarding Process

The primary objective of the six-stage workflow is cognitive scaffolding. Newcomers frequently experience hesitation not because they cannot write code, but because they are uncertain about repository norms, branch naming, and pull-request etiquette. Decomposing the process into explicit checklists provides reassurance and structured guidance.

VIII. THREATS TO VALIDITY

- **Construct Validity:** The use of label heuristics as proxies for issue difficulty may not perfectly capture actual implementation complexity, as repository maintainers do not apply labels uniformly.
- **Internal Validity:** Although end-to-end system validation demonstrates that all platform components operate reliably, no controlled human-subject trial has yet measured the comparative onboarding velocity of developers using the platform versus traditional GitHub interfaces.
- **External Validity:** GitHub repositories exhibit diverse documentation practices and maintenance conventions. The effectiveness of label-based ingestion may vary across ecosystems outside JavaScript and Python.
- **Data and API Validity:** The platform relies on public GitHub REST endpoints; API schema modifications or service downtime can affect ingestion availability.

IX. ETHICAL CONSIDERATIONS

Open Source Companion exclusively processes publicly available metadata from GitHub in compliance with GitHub’s Developer Terms of Service. Contributor activity data extracted during OAuth authentication is utilized solely for localized skill modeling within the authenticated session, without redistribution or commercial profiling. Furthermore, transparent heuristic criteria ensure that task difficulty categorization remains fully inspectable by contributors.

X. CONCLUSION AND FUTURE WORK

This paper presented *Open Source Companion*, an intelligent, full-stack platform designed to facilitate newcomer onboarding in open-source software projects. The platform addresses fragmented entry barriers by coupling automated issue acquisition from GitHub with interpretable difficulty categorization, automated OAuth skill profiling, a six-stage guided contribution workspace, and gamification feedback. System validation confirms that the implemented Next.js, PostgreSQL, and Prisma architecture functions cohesively under containerized deployment.

Future work will implement the proposed machine-learning roadmap: replacing heuristic categorization with supervised tree-based ensembles (XGBoost/Random Forest), introducing Sentence-BERT and FAISS for dense semantic issue retrieval, and executing the empirical benchmark protocol alongside a longitudinal user study to evaluate newcomer retention.

REFERENCES

- [1] A. Mockus, R. T. Fielding, and J. D. Herbsleb, “Two case studies of open source software development: Apache and Mozilla,” *ACM Transactions on Software Engineering and Methodology*, vol. 11, no. 3, pp. 309–346, 2002.
- [2] I. Steinmacher, M. A. G. Silva, M. A. Gerosa, and D. F. Redmiles, “A systematic literature review on the barriers faced by newcomers to open source software projects,” *Information and Software Technology*, vol. 59, pp. 67–85, 2015.
- [3] B. Dagenais, H. Ossher, R. K. E. Bellamy, M. P. Robillard, and J. P. de Vries, “Moving into a new software project landscape,” in *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering (ICSE)*, 2010, pp. 275–284.
- [4] X. Tan, M. Zhou, and Z. Sun, “A first look at good first issues on GitHub,” in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2020, pp. 398–409.
- [5] W. Xiao, H. He, W. Xu, X. Tan, J. Dong, and M. Zhou, “Recommending good first issues in GitHub OSS projects,” in *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, 2022, pp. 1830–1842.
- [6] M.-A. Storey, A. Zagalsky, F. F. Filho, L. Singer, and D. M. German, “How social and communication channels shape and challenge a participatory culture in software development,” *IEEE Transactions on Software Engineering*, vol. 43, no. 2, pp. 185–204, 2017.
- [7] I. Steinmacher, T. U. Conte, M. A. Gerosa, and D. F. Redmiles, “Overcoming open source project entry barriers with a portal for newcomers,” in *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, 2016, pp. 273–284.
- [8] J. Anvik, L. Hiew, and G. C. Murphy, “Who should fix this bug?” in *Proceedings of the 28th International Conference on Software Engineering (ICSE)*, 2006, pp. 361–370.
- [9] D. Čubranić, G. C. Murphy, J. Singer, and K. S. Booth, “Hipikat: A project memory for software development,” *IEEE Transactions on Software Engineering*, vol. 31, no. 6, pp. 446–465, 2005.
- [10] H. He, H. Su, W. Xiao, R. He, and M. Zhou, “Personalized first issue recommender for newcomers in open source projects,” in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023, pp. 800–812.
- [11] R. Kikas, M. Dumas, and D. Pfahl, “Using dynamic and contextual features to predict issue lifetime in GitHub projects,” in *Proceedings of the 13th International Conference on Mining Software Repositories (MSR)*, 2016, pp. 291–302.
- [12] J. Zhou, H. Zhang, and D. Lo, “Where should the bugs be fixed? More accurate information retrieval-based bug localization based on bug reports,” in *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, 2012, pp. 14–24.
- [13] O. Pedreira, F. García, N. Brisaboa, and M. Piattini, “Gamification in software engineering – A systematic mapping,” *Information and Software Technology*, vol. 57, pp. 157–168, 2015.
- [14] B. Vasilescu, A. Serebrenik, P. Devanbu, and V. Filkov, “How social Q&A sites are changing knowledge sharing in open source software communities,” in *Proceedings of the 17th ACM Conference on Computer Supported Cooperative Work & Social Computing (CSCW)*, 2014, pp. 342–354.
- [15] L. Singer and K. Schneider, “It was a bit of a race: Gamification of version control,” in *Proceedings of the 2nd International Workshop on Games and Software Engineering (GAS)*, 2012, pp. 5–8.
- [16] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785–794.
- [17] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [18] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.
- [19] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.